

refactorings. This is a long term phase that the DRIs of the domain need to nurture over time. We will use Packwerk failures and the dependency diagram to influence the modular design.

- Revisit whether a class should be private instead of public, and create a better interface.
- Move constants to different package if too coupled with that.
- Join packages if they are too coupled to each other.

Once we have Packwerk configured for the Rails application (step 2 above), emerging domains could be directly implemented as Packwerk packages, benefiting from isolation and clear interface immediately.

SECTION: engineering/architecture/design-documents/modular_monolith/proof_of_concepts.md

Modularization of our monolith is a complex project. There will be many unknowns. One thing that can help us mitigate the risks and deliver key insights are Proof-of-Concepts that we could deliver early on, to better understand what will need to be done.

Inter-module communication

A PoC that we plan to deliver is a PoC of inter-module communication. We do recognize the need to separate modules, but still allow them to communicate together using a well defined interface. Modules can communicate through a facade classes (like libraries usually do), or through eventing system. Both ways are important.

The main question is: how do we want to define the interface and how to design the communication channels?

It is one of our goals to make it possible to plug modules out, and operate some of them as separate services. This will make it easier to deploy GitLab.com in the future and scale key domains. One possible way to achieve this goal would be to design the inter-module communication using a protobuf as an interface and gRPC as a communication channel. When modules are plugged-in, we would bypass gRPC and serialization and use in-process communication primitives (while still using protobuf as an interface). When a module gets plugged-out, gRPC would carry messages between modules.

Use Packwerk to enforce module boundaries

Packwerk is a static analyzer that helps defining and enforcing module boundaries in Ruby.

In this PoC merge request we demonstrate a possible directory structure of the monolith broken down into separate modules.

The PoC also aims to solve the problem of EE extensions (and JH too) allowing the Rails autoloader to be tweaked depending on whether to load only the Core codebase or any extensions.

The PoC also attempted to only move a small part of the `Ci::` namespace into a `components/ci` Packwerk package. This seems to be the most iterative approach explored so far.

There are different approaches we could use to adopt Packwerk. Other PoC's also explored are the large extraction of CI package and moving the 2 main CI classes into a package.

All 3 PoC's have a lot in common, from the introduction of Packwerk packages and configurations to setting paths for the autoloader to work with any packages. What changes between the various merge requests is the approach on choosing which files to move first.

The main goals of the PoC were:

- understand if Packwerk can be used on the GitLab codebase.
- understand the learning curve for developers.
- verify support for EE and JH extensions.
- allow gradual modularization.

Positive results

- Using Packwerk would be pretty simple on GitLab since it's designed primarily to work on Rails codebases.
- We can change the organization of the domain code to be module-oriented instead of following the MVC pattern. It requires small initial changes to allow the Rails autoloading to support the new directory structure, which is by the way not imposed by Packwerk. After that, registering a new top-level package/bounded-context would be a 1 LOC change.
- Using the correct directory structure indicated in the PoC allows packages to contain all the code, including EE and JH extensions.
- Gradual modularization is possible and we can have any degree of modularization as we want, from initial no enforcement down to complete isolation simulating an in-memory micro-service environment.
- Moving files into a Packwerk package doesn't necessarily mean renaming constants. While this is not advisable long term, it's an extra flexibility that the tool provides.
 - For example: If we are extracting the `Ci::` module into a Packwerk package there can be constants that belong to the CI domain but are not namespaced, like `CommitStatus` or that have a different namespace, like `Gitlab::Ci::`. Packwerk allows such constants to be moved inside the `ci` package and correctly flags boundary violations.
- Packwerk enhancements from RubyAtScale tooling allow to enforce that all constants inside a package share the same Ruby namespace. We eventually would want to leverage that.
- RubyAtScale provides also tools to track metrics about modularization and adoption which we would need to monitor and drive as an engineering organization.
- Packwerk has IDE extensions (e.g. for VS Code) to provide realtime feedback on violations (like RuboCop). It can also be run via CLI during the development workflow against a single package. It could be integrated into pre-push Git hooks or Danger during code reviews.

Challenges

Some of these challenges are not specific to Packwerk as tool/approach. They were observed during the PoC and are more generically related to the process of modularization:

- There is no right or wrong approach when introducing Packwerk packages. We need to define clear guidelines to give developers the tools to make the best decision:
 - Sometimes it could be creating an empty package and move files in it gradually.
 - Sometimes it could be wrapping an already well designed and isolated part of the codebase.
 - Sometimes it could be creating a new package from scratch.
- As we move code to a different directory structure we need to involve JiHu as they manage extensions following the current directory structure.
We may have modules that are partially migrated and we need to ensure JiHu is up-to-date with the current progresses.
- After privacy/dependency checks are enabled, Packwerk will log a lot of violations (like RuboCop TODOs) since constant references in a Rails codebase are very entangled.
 - The team owning the package needs to define a vision for the package.
What would the package look like once all violations have been fixed?
This may mean specifying where the package fits in the context map
of the system. How the current package should be used by another package A and how it should use other packages.
 - The vision above should tell developers how they should fix these violations over time.
Should they make a specific constant public? Should the package list another package as its dependencies? Should events be used in some scenarios?
 - Teams will likely need guidance in doing that. We may need to have a team of engineers, like maintainers with a very broad understanding of the domains, that will support engineering teams in this effort.
- Changes to CI configurations on tuning Knapsack and selective testing were ignored during the PoC.

Frontend sorting hat

Frontend sorting-hat is a PoC for combining multiple domains to render a full page of GitLab (with menus, and items that come from multiple separate domains).

Frontend assets aggregation

Frontend assets aggregation is a PoC for a possible separation of micro-frontends.

SECTION: engineering/architecture/design-documents/modular_monolith/references.md

Related design docs

- Composable codebase design doc

Related Issues

- Split GitLab monolith into components
- Make it simple to build and use "Decoupled Services"
- Use nested structure to organize CI classes
- Create new models / classes within a module / namespace
- Make teams to be maintainers of their code
- Add backend guide for Dependency Injection

Internal Slack Channels

- modularmonolith
- architecture

Presentations

- The Myth of the Modular Monolith - Rails 2024 Keynote by Eileen Uchitelle

Reference Implementations / Guides

Gusto / RubyAtScale:

- RubyAtScale toolchain for modularization
- Gusto's engineering blog
- Gradual modularization (successor to CBRA)
- Component-Based Rails Applications ("deprecated")

Shopify:

- Packwerk
- Shopify's journey to modularization
- Internal GitLab doc transcript of an AMA with a Shopify engineer

Domain-Driven Rails / Rails Event Store:

Rails Event Store is relevant because it is a mechanism to achieve many of the goals discussed here, and is based upon patterns used by Arkency to build production applications.

This doesn't mean we need to use this specific framework or approach.

However, the general concepts of DDD/ES/CQRS are important and in some cases maybe necessary to achieve the goals of this blueprint, so it's useful to have concrete production-proven implementations of those concepts to look at as an example.

- Arkency's domain-driven Rails
- Arkency's Rails Event Store

App Continuum:

An illustration of how an application can evolve from a small, unstructured app, through various stages including a modular well-structured monolith, all the way to a microservices architecture.

Includes discussion of why you might want to stop at various stages, and specifically the challenges/concerns with making the jump to microservices, and why sticking with a well-structured monolith may be preferable in many cases.

- App Continuum

SECTION: engineering/architecture/design-
documents/modular_monolith/decisions/001_modular_application_domain.md

Context

Before we modularize a codebase we first needed to define how we are going to divide it.

Decision

We start by focusing on the application domain (backend business logic) leaving the application adapters (Web controllers and views, REST/GraphQL endpoints) outside the scope of the modularization initially.

The reasons for this are:

1. Code in application adapters may not always align with a specific domain. For example: a project settings endpoint or a merge request page contain references to many domains.
1. There was a need to run separate Rails nodes for the SaaS architecture using different profiles in order to save on memory.
For example: on SaaS we wanted to be able to spin up more Sidekiq nodes without the need to load the whole Rails application. The assumption is that for running Sidekiq we don't need ActionCable, REST endpoints, GraphQL mutations or Rails views.
We only need the application domain and infrastructure code.
This could still be true even with the introduction of Cells but we need to re-evaluate this assumption.
1. Keep the scope and effort smaller. Tackling only domain code is easier to understand than the complexity of how to breakdown the application adapters and all their edge cases.

The decision to scope out application adapters is not final and we decided to defer it to later.

Finally, the infrastructure code containing technical concerns (typically the lib/) will be part of a common "platform" module that every domain module will depend on in order to function.

The "platform" module can be broken down into independent libraries extracted as gems.

Consequences

We focus on modularizing business logic primarily we simplify the rules and guidelines for engineers. We can apply the same set of patterns across modules.

Alternatives

We looked into including application adapters to the modularization effort but noticed that:

1. Modularizing adapters is more delicate as we need to preserve user-facing dependencies like routes.
1. The size of the adapters code is much smaller than the whole application domain.

SECTION: engineering/architecture/design-
documents/modular_monolith/decisions/002_bounded_contexts_definition.md

Context

With the focus primarily on the application domain we needed to define how to modularize it.

Decision

The application domain is divided into bounded contexts which define the top-level modules of GitLab application. The term bounded context is widely used in Domain-Driven Design.

Defining bounded contexts means to organize the code around product structure rather than organizational structure.

From the research in Proposal: split GitLab monolith into components it seems that following product categories, as a guideline, would be much better than translating organization structure into folder structure (for example, app/modules/verify/pipeline-execution/...).

However, this guideline alone is not sufficient and we need a more specific strategy:

- Bounded contexts (top level modules) should be sufficiently deep to encapsulate implementation details and provide a smaller interface.
- Some product categories, such as Browser Performance Testing, are just too small to represent a bounded context on their own.
We should have a strategy for grouping product categories together when makes sense.
- Product categories don't necessarily translate into clean boundaries.
Category:Pipeline Composition and Category:Continuous Integration are some examples where Pipeline Authoring team and Pipeline Execution team share a lot of code.
- Some parts of the code might not have a clear product category associated to it.

Despite the above, product categories provide a rough view of the bounded contexts at play in

the application.

For that we use product categories to sketch the initial set of bounded contexts.

Then, group related or strongly coupled categories under the same bounded context and create new bounded contexts if missing.

Consequences

In May 2024 we completed the Bounded Contexts working group which completed the first phase of modularization, described in this page.

We defined a list of bounded contexts in code and started enforcing them with RuboCop, in order to move towards a fully namespaced monolith. Team members can edit this list by creating and deleting bounded contexts explicitly and the decision is reviewed by Staff+ engineers.

Alternatives

We evaluated whether to align the code to the organizational structure but we decided it wasn't viable:

- Product categories can change ownership and we have seen some pretty frequent changes, even back and forth.

Moving code every time a product category changes ownership adds too much maintenance overhead.

- Teams and organization changes should just mean relabelling the ownership of specific modules.

- Coupling and complexity are directly correlated to business logic and product structure.

A code organization that aligns to organizational structure could generate unnecessary complexity and much more coupling.

SECTION: engineering/architecture/design-documents/modular_monolith/decisions/003_stewardship.md

Context

How do we assign stewardship to domain and platform modules? We have a large amount of shared code that does not have explicit stewards who can provide a vision and direction on that part of code.

Decision

We use the term stewards instead of owners to be more in line with GitLab principle of everyone can contribute. Stewards are care takers of the code. They know how a specific functionality is designed and why. They know the architectural characteristics and constraints. However, they welcome changes and guide contributors towards success.

A module, whether is from a domain bounded context or platform module, must have at least 1 group of stewards.

This group can be a team name (or GitLab group handle). Optionally, the list of stewards can include single IC entries.

When we will use a Packwerk package to extract a module we will be able to indicate stewardship directly in the package.yml:

```
yaml
metadata:
  stewards:
    - group::pipeline execution team name
    - group::pipeline authoring team name
    - @grzesiek IC
    - @ayufan IC
```

For platform modules (e.g. Gitlab::Redis) we might not have a whole team dedicated as stewards since all platform code is classified as "shared". However, team members can add themselves as experts of a particular functionality.

Consequences

Stewardship defined in code can be very powerful:

- Sections of CODEOWNERS could be automatically generated from packages' metadata.
- Review Roulette or Suggested Reviews features can use this list as first preference.
- Engineers can easily identify stewards and have design conversations early.
- Gems living in the monolith (gems/), which should be wrapped into a Packwerk package, can benefit of having explicit stewards.

Alternatives

In the initial phase of modularization, before adopting Packwerk, we don't have an explicit concept of ownership. We are initially relying on each team to know what bounded contexts they are responsible for. For the "shared code" in the platform modules we initially expect maintainers to fill the role of stewards.

- Pros: we give trainee maintainer a clear development path and goals. Today it feels unclear what they must learn in order to become successful maintainers.
- Cons: The amount of "shared" code is very large and still hard to understand who knows best about

a particular functionality. Even extracting code into gems doesn't solve the lack of explicit ownership.

SECTION: engineering/architecture/design-
documents/modular_monolith/hexagonal_monolith/index.md

Background

This design document supersedes the previous Composable GitLab Codebase where we explored the idea of separating the codebase into technical runtime profiles: for example, run the monolith solely as a Sidekiq node.

With a modular monolith and the use of an Hexagonal Architecture, we can achieve both separation of domains as well as separation of application adapters, which may include the usage of engines and/or different runtime profiles.

Summary

TL;DR: Change the Rails monolith from a big ball of mud state to a modular monolith

that uses an Hexagonal architecture) (or ports and adapters architecture).

Extract cohesive functional domains into separate directory structure using Domain-Driven Design practices.

Extract infrastructure code (logging, database tools, instrumentation, etc.) into gems, essentially remove the need for lib/ directory.

Define what parts of the functional domains (for example application services) are of public use for integration (the ports)

and what parts are instead private encapsulated details.

Define Web, Sidekiq, REST, GraphQL, and Action Cable as the adapters in the external layer of the architecture.

Use Packwerk to enforce privacy and dependency between modules of the monolith.

Details

mermaid

flowchart TD

u([User]) -- interacts directly with --> AA[Application Adapter: WebUI, REST, GraphQL, git, ...]

AA --uses abstractions from--> D[Application Domain]

AA -- depends on --> Platform

D -- depends on --> Platform[Platform: gems, configs, framework, ...]

Application domain

The application core (functional domains) is composed of all the code that describes the business logic, policies and data

that is unique to GitLab product.

It is divided into separate top-level bounded contexts.

A bounded-context is represented in the form of a Ruby module.

This follows the existing guideline on naming namespaces but puts more structure to it.

Modules should:

- Be deep enough to encapsulate a lot of the internal logic, state and data.
- Have a public interface that is as small as possible, safe to use by other bounded contexts and well documented.
- Be cohesive and represent the SSoT (single source of truth) of the feature it describes.

Feature categories represent a product area that is large enough for the module to be deep, so we don't have a proliferation of small top-level modules. It also helps the codebase to follow the ubiquitous language.

A team can be responsible for multiple feature categories, hence owning the vision for multiple bounded contexts.

While feature categories can sometimes change ownership, this change of mapping the bounded context to new owners is very cheap.

Using feature categories also helps new contributors, either as GitLab team members or members of the wider community, to navigate the codebase.

If multiple feature categories are strongly related, they may be grouped under a single bounded context.

If a feature category is only relevant in the context of a parent feature category, it may be included in the parent's bounded context. For example: Build artifacts existing in the context of Continuous Integration feature category and they may be merged under a single bounded context.

The application domain has no knowledge of outer layers like the application adapters and only depends on the platform code. This makes the domain code to be the SSoT of the business logic, be reusable and testable regardless whether the request came from the WebUI or REST API.

If a dependency between an outer layer and an inner layer is required (domain code depending on the interface of an adapter), this can be solved using inversion of control techniques, especially dependency injection.

Application adapters

>>>

Adapters are the glue between components and the outside world.

They tailor the exchanges between the external world and the ports that represent the requirements of the inside

of the application component. There can be several adapters for one port, for example, data can be provided by

a user through a GUI or a command-line interface, by an automated data source, or by test scripts. -

WikipediaPrinciple)

>>>

Application adapters would be:

- Web UI (Rails controllers, view, JS and Vue client)
- REST API endpoints
- GraphQL Endpoints

They are responsible for the interaction with the user. Each adapter should interpret the request, parse parameters and invoke the right abstraction from the application domain, then present the result back to the user.

Presentation logic, and possibly authentication, would be specific to the adapters layer.

The application adapters layer depends on the platform code to run: the Rails framework, the gems that power the adapter, the configurations and utilities.

Platform code

For platform code we consider any classes and modules that are required by the application domain and/or application adapters to work.

The Rails' lib/ directory today contains multiple categories of code that could live somewhere else, most of which is platform code:

- REST API endpoints could be part of the application adapters.
- domain code (both large domain code such as Gitlab::Ci and small such as Gitlab::JiraImport) should be moved inside the application domain.
- The rest could be extracted as separate single-purpose gems under the gems/ directory inside the monolith.
This can include utilities such as logging, error reporting and metrics, rate limiters, infrastructure code like Gitlab::ApplicationRateLimiter, Gitlab::Redis, Gitlab::Database and generic subdomains like Banzai.

Base classes to extend Rails framework such as ApplicationRecord or ApplicationWorker as well as GitLab base classes such as BaseService could be implemented as gem extensions.

This means that aside from the Rails framework code, the rest of the platform code resides in gems/.

Eventually all code inside gems/ could potentially be extracted in a separate repository or open sourced.

Placing platform code inside gems/ makes it clear that its purpose is to serve the application code.

Enforcing boundaries

Ruby does not have the concept of privacy of constants in a given module. Unlike other programming languages, even extracting well documented gems doesn't prevent other developers from coupling code to implementation details because all constants are public in Ruby.

We can have a codebase perfectly organized in an hexagonal architecture but still having the application domain, the biggest part of the codebase, being a non modularized big ball of mud.

Enforcing boundaries is also vital to maintaining the structure long term. We don't want that after a big modularization effort we slowly fall back into a big ball of mud gain by violating the boundaries.

We explored the idea of using Packwerk in a proof of concept to enforce module boundaries.

Packwerk is a static analyzer that allows to gradually introduce packages in the codebase and enforce privacy and explicit dependencies. Packwerk can detect if some Ruby code is using private implementation details of another package or if it's using a package that wasn't declared explicitly as a dependency.

Being a static analyzer it does not affect code execution, meaning that introducing Packwerk is safe and can be done gradually.

Companies like Gusto have been developing and maintaining a list of development and engineering tools for organizations that want to move to using a Rails modular monolith around Packwerk.

EE and JH extensions

One of the unique challenges of modularizing the GitLab codebase is the presence of EE extensions (managed by GitLab) and JH extensions (managed by JiHu).

By moving related domain code (e.g. Ci::) under the same bounded context and Packwerk package, we would also need to move ee/ extensions in it.

To have top-level bounded contexts to also match Packwerk packages it means that all code related to a specific domain

needs to be placed under the same package directory, including EE extensions, for example.

The following is just an example of a possible directory structure:

```
shell
domains
... ci
. ... package.yml      package definition.
. ... packwerk.yml     tool configurations for this package.
. ... packagetodo.yml  existing violations.
. ... core             Core features available in Community Edition and always autoloaded.
. . ... app
. . . ... models/...
. . . ... services/...
. . . ... lib/...      domain-specific lib moved inside app together with other classes.
. . ... spec
. . ... models/...
. ... ee              EE extensions specific to the bounded context, conditionally
autoloaded.
. . ... models/...
. . ... spec
. . ... models/...
. ... public          Public constants are placed here so they can be referenced by other
packages.
. ... core
. . ... app
. . . ... models/...
. . ... spec
. . ... models/...
. ... ee
. ... app
. . ... models/...
. ... spec
. ... models/...
... mergerequests/
... repositories/
... ..
```

Challenges

- Such changes require a shift in the development mindset to understand the benefits of the modular architecture and not fallback into legacy practices.
- Changing the application architecture is a challenging task. It takes time, resources and commitment but most importantly it requires buy-in from engineers.
- This may require us to have a medium-long term team of engineers or a Working Group that makes progresses on the architecture evolution plan, foster discussions in various engineering channels and

resolve adoption challenges.

- We need to ensure we build standards and guidelines and not silos.
- We need to ensure we have clear guidelines on where new code should be placed. We must not recreate junk drawer folders like lib/.

Opportunities

The move to a modular monolith architecture enables a lot of opportunities that we could explore in the future:

- We could align the concept of domain expert with explicitly owning specific modules of the monolith.
- The use of static analysis tool (such as Packwerk, RuboCop) can catch design violations in development and CI, ensuring that best practices are honored.
- By defining dependencies between modules explicitly we could speed up CI by testing only the parts that are affected by the changes.
- Such modular architecture could help to further decompose modules into separate services if needed.

SECTION: engineering/architecture/design-documents/new_auth_stack/_index.md

{{< engineering/design-document-header >}}

Summary

GitLab authentication and authorization code has been evolving organically for more than decade so far. Most of the authentication and authorization mechanisms live inside GitLab Rails monolith, with hundreds of software engineers contributing to the project each day. This results in code complexity growing:

1. We have 10+ different token types used for authentication.
1. We have 5+ different ways to authenticate a user session.
1. We have 3+ ways to use GitLab as an identity provider.
1. We use a tightly-coupled and complex Declarative Policies language built by us.
1. Our authorization tooling is causing frequent performance-related incidents.
1. Teams are building more of their own authentication methods in silo.

Today GitLab is a mission-critical component for many customers, who rely on our authentication and authorization mechanisms to also grant or reject access to their infrastructure.

Based on the challenges listed above we decided to focus on building the GitLab Adaptive Trust Environment (GATE), following three core principles aligned under a single overarching technical strategy aimed to consolidate and centralize our core authentication and authorization mechanisms:

1. Zero Trust: Never trust, always verify. Access is granted based on verified identity, context, and explicit policy. Verify trust continuously, on every level of application stack.
1. Least Privilege: Grant the minimum necessary permissions for the shortest possible time required to complete a task for all tokens and authenticated principals.
1. Ambient Security: Authentication, authorization and continuous verification are always present. Most engineers and customers don't have to deal with this complexity: it is largely abstracted away and enabled by default.

To provide more clarity around how to deliver GATE we will focus on building three key features anchoring this redesign in us delivering customer value innovation:

1. GitLab Workload Identity Federation: built to reduce reliance on Personal Access Tokens, and long-lived credentials.
1. Ambient Credentials in GitLab CI: significantly reducing the need of using tokens / credentials / keys in CI jobs.
1. Fine-grained access controls for all authenticated principals: to improve security posture across the whole feature set.

Details

The architectural decisions register and more details can be found in the private project in The New Auth Stack Design Doc.

The main GitLab epic with issues and workstreams is Deliver The New Auth Stack in GitLab.

SECTION: engineering/architecture/design-documents/notes_table_partitioning/_index.md

{{< engineering/design-document-header >}}

Problem

The notes table is one of the largest PostgreSQL DB tables in GitLab, whose size in the .com production database exceeded 1.5TB as of Nov 2023, increasingly endangering the reliability of GitLab.com and large self-managed instances.

Available partitioning or sharding methods must be evaluated for the table and executed as early as possible to meet the 100GB per table target size limit.

Overview of notes table

The table composition as of Nov 2023

The majority of the records in the table were for merge requests.

Noteable type	% of total records	Num. records
Merge Request	67%	1002272868
Issue	23%	348020507
Commit	<~ 5 %	67790930
Epic	<~ 0.05 %	6196244
Others	<~ 5%	
Total		1488612100

The note and notehtml columns took up 183GB and 580GB respectively, occupying ~77% of the storage space used by the table and its supporting indexes.

note stores the raw note texts and notehtml caches the HTML renders of the raw note texts.

Column	Size (GB)	% of Total
note	183 GB	16%
notehtml	580 GB	51%
MR related columns	94 GB	0.8%
Other columns	383 GB	24%
Total	1,240 GB	~100%

The indexes on the table took up the remaining 300 GB or so.

Design of the notes table

notes has polymorphic associations through the following three columns.

- noteabletype: stores the type of a noteable, for example Issue, MergeRequest, Commit.
- noteableid: stores the ID of a noteable.
- commitid: stores the Git SHA of a commit.

When a note's noteabletype is Commit, noteableid is NULL and commitid is used to reference the commit.

The associated models are: MergeRequest, Vulnerability, Epic, Snippet, Commit, DesignManagement::Design, Issue, AlertManagement::Alert and AbuseReport

All notes should belong to a namespace through noteable except for abuse notes that are instance-level.

Reducing contention on the lockmanager lwlocks

The polymorphic associations for the table have an important implication for partitioning.

Partition must be done in a way that the queries targeting the table accesses the minimum number of partitions or in a manner that does not deplete the 16 fastpath locks to

reduce contention on the lockmanager lwlocks.

For example, the most common access pattern for the notes table is fetching the notes for a noteable such as Issue.

If the table were to be partitioned by id (range, hash or list), the following query could end up hitting

all partitions: `SELECT FROM notes WHERE noteabletype='Issue' AND noteableid=1;`

Suppose we needed to get notes by ids to display user activities or TODOs instead:

`SELECT FROM todos INNER JOIN notes WHERE notes.id=todos.noteid AND todos.id IN (1, 2, 3);`

PostgreSQL would be able to prune out the partitions that didn't contain the noteids and only access

the minimum partitions needed to execute the query.

It's challenging that many columns in the notes table are nullable including the columns used for polymorphic associations as well as namespaces and projects while partitioning usually requires partitioning columns to be non-nullable.

Partitioning methods

Here we consider various options for partitioning and restructuring the notes table.

1. Split the table by domain model

Splitting into separate tables by domain models (issues, MRs, epics)

aligns with best practices but the resulting tables would still exceed 100GB.

For example, we could have separate `issuenotes`, `mergerequestnotes` and `epicnotes` tables.

Benefits:

- Aligns with domain models and the composition of the table
 - There are many merge request specific columns that interfere with applying other partitioning strategies,
- and contribute to index bloat and non-ideal data alignments.
- Addresses polymorphic associations and constraints issues
 - On top of having polymorphic associations which are discouraged,
- the presence of the `commitid` column storing Git SHA hashes prevents utilizing database constraints fully.

Drawbacks:

- Significant code changes would be required across the codebase
- Even after splitting by domain,
 - the resulting domain tables would exceed the 100GB target size limit and each table would need to be partitioned.

2. Partition by hash using namespaceid

Given the typical access pattern of retrieving notes for a noteable, we might consider using noteabletype, noteableid, and commitid as hash key columns. However, some queries like this one used to preload notes work with ids, namespaceid could be the better choice as the sole hash key

Benefits:

- We can achieve the 100GB target size limit without additional work although some partitions may grow at a faster rate in the future.

Drawbacks:

- The primary key of a hash partitioned table must include the hash key columns, but all proposed hash keys here are nullable, and thus cannot be part of the primary key and lose referential integrity. However, this drawback only exists when partitioning by the polymorphic association columns. namespaceid would soon become the sharding key for the notes table for the Cells 1.0.
- namespaceid could be sharding keys for all the tables referencing the notes table allowing us to easily add foreign keys to the partitioned notes table as the work for Cells 1.0 progresses.
- Some code change is still required to update all the notes queries to include namespaceid.
- This method does not address any of the existing structural problems like polymorphic associations or too many merge request specific columns (and the consequences)

3. Vertical split of the table

A vertical split of the two largest columns note and notehtml or just notehtml into a separate table can reduce overall storage needs.

It's worth noting that the notes table cannot significantly benefit from PostgreSQL's TOAST feature.

Most note texts (note column) do not exceed the 2kB default threshold necessary to trigger compression and OUT-OF-LINE storage.

The table for the vertically split column(s) could be partitioned by int range using the notes table's id column.

Benefits:

- Vertical partitioning of note and notehtml columns can improve the table layout and compact the tuple sizes of the remaining notes table to allow for better spatial locality.
- It might be viewed as a more incremental approach as the notes table would remain in place.

Drawbacks:

- In order to avoid the lock contention problem, a batching strategy must be implemented when preloading the table containing the vertically split columns.
Suppose there are 16+ partitions. The first partition contains the data for the notes records with IDs <100.
The second partition contains the data for the notes records with IDs >=100 and so on.
To make sure a preload query does not access too many partitions, we can break it into several queries:
`SELECT FROM pnotesdata WHERE noteid < 100 AND noteid IN (1, 100, 20000, 30000),`
`SELECT FROM pnotesdata WHERE noteid >= 20000 AND noteid IN (1, 100, 20000, 30000) and so on.`
- CacheMarkdownField, the concern that notes uses, has implicit dependencies on other parts of the codebase.
The attempts to override them or transparently delegate the related methods do not work cleanly or easily.
Also see the section on Markdown caching
- Additional partitioning work for the notes table is still required to meet the 100GB target size limit.
- This method does not address any of the existing structural problems like polymorphic associations or too many merge request specific columns (and the consequences)

Notes on DB-based Markdown caching

Merge request notes follow the typical decay pattern where the notes start losing relevance once their merge request become closed. The same decay pattern may be applicable to other noteable types and dropping the cached Markdown for the notes older than some period could be a viable method to reduce the stored data.

One possible risk is that re-calculating older notes whose cached Markdown was dropped could have a thrashing effect on both the Rails application and the PostgreSQL hosts. In the past, it's been observed that rendering many notes and updating the cache severely strained the application and the database whenever the cached Markdown version was bumped for notes.

It's worth investigating caching Markdown solely to Redis and removing the database caching layer if the thrashing effect turns out to be less concerning.

4. Partition by list using noteabletype

We may consider partitioning the notes table itself by the values of the noteabletype column.

NOTE:

As of Mar 8th 2024, there is a INVALID non-null check constraint on noteabletype. A small number of notes records without noteabletype have been found and removed from

GitLab.com's production database.

The non-null check constraint is planned to be validated in 17.0

Benefits:

- Almost all notes queries contain noteabletype and that makes the column an ideal choice to use when partitioning by list.
- Partitioning by list with noteabletype can allow us to further partition the resulting partitions by domain.

Drawbacks:

- Sub-partitioning still requires partitioning keys to be present in the root table's primary key.
- This approach requires adding many foreign keys to the tables referencing the notes table. The partitioned table would use a composite primary key (noteabletype, id) so noteabletype would need to be first added to all the referencing tables and backfilled.
- More code change might be required to ensure Active Record models work with the partitioned table that uses the composite primary key.
- This method does not address any of the existing structural problems like polymorphic associations or too many merge request specific columns. However, the possibility of sub-partitioning by domain does dampen the drawback.

SECTION: engineering/architecture/design-documents/notifications/_index.md

<!-- Design Documents often contain forward-looking statements -->

<!-- vale gitlab.FutureTense = NO -->

<!-- This renders the design document header on the detail page, so don't remove it-->

{{< engineering/design-document-header >}}

<!--

Don't add a h1 headline. It'll be added automatically from the title front matter attribute.

For long pages, consider creating a table of contents.

-->

Summary

The current notification system is based on emails, requiring users to move constantly between their inbox and GitLab. At the same time, we have a system of to-dos that partially overlaps with notifications but generally establishes only a subset of those. To-dos are also not

controlled by any user preferences.

To improve user experience and create parity between to-dos and email notifications, we propose creating a notification system within GitLab itself, which will replace our current, fixed system of to-dos. The system of notifications will be based on how we handle e-mail notifications now - users will be able to define rules for email notifications and notification visible in the notification center in GitLab. Visually, notification center will be similar to our current to-dos page.

Epic that defines needs for the new system.

Motivation

Our goal is to create a system that allows users to receive information about activity in their groups and projects in GitLab itself, without the need to check their email inbox. These notifications should be personalizable, easy to filter, and simple to mark as done/read. We want to grow the usage of notifications (metrics to be determined).

Our internal motivation is to create a system that is easy to extend and loosely coupled with other parts of the codebase. We also want to create a system with clear retention policies, allowing for a maintainable database load. We also want to ensure that adding new notifications to the codebase is easy and feature teams can self-serve with it.

Goals

- Create a comprehensive experience for users, who will have one place to see everything they need to know to work
- Improve MAU (Monthly Active Users) metric between current to-dos and new notification center
- Refactor code to achieve higher cohesion and a more extendable system

Non-Goals

- Replace all functionality that users may get from their inbox (ie. search all mentions of all time). We instead want to focus on day to day interactions mostly using GitLab

Proposal

Summary

We propose unifying our currently fragmented notification systems (email notifications and to-dos) into a single, cohesive notification center using an event-driven architecture. This will improve user experience, simplify code maintenance, and enable future extensibility.

Proposed Solution

Create a unified notification center built on an event-driven architecture that:

- Uses our existing Event Store as the backbone for all notifications
- Establishes a single database model for all notification types
- Implements sensible retention policies (3-6 months by default)

- Allows users to save important notifications indefinitely
- Provides consistent API access for both internal and external integrations

Pros and Cons

Pros

1. Improved user experience:
 - Single location for all notifications
 - Consistent interface and behavior
 - Greater control over notification preferences
2. Technical benefits:
 - Decoupled architecture allows independent service development
 - Centralized notification logic improves maintainability
 - Event-driven design enables easy addition of new notification types
 - Reduced code duplication and complexity
3. Future flexibility:
 - Straightforward path to add new notification channels (web, mobile push, etc.)
 - API-first approach enables third-party integrations
 - Retention policies prevent database bloat

Cons

1. Migration complexity:
 - Requires careful handling of existing notifications
 - User settings need thoughtful transition
 - Temporary increased system complexity during transition
2. System dependencies:
 - Increased reliance on Event Store reliability
 - Need to manage event processing performance
 - Potential for more complex failure scenarios
 - Bigger database load due to more notifications being saved
3. Resource requirements:
 - Significant engineering effort
 - Careful testing required to ensure seamless transition

Design and implementation details

Database table

We need to persist notifications in a new notifications database table. This table will store data about notifications - what kind of notification it is, what resource it's connected with, the state of it, information if it is saved by the user, etc.

An example of the table schema, models, and service implementation can be found in `001databaseschema.md`.

Our requirements are that we will be accessing todos in the majority of situations by userid, and possible filtering patterns are:

- by project
- by group (so all notifications coming from projects from particular group)
- by author
- by resource connected to the notification
- by action (for example: mentioned/assigned/etc)
- by state
- by snoozed status.

We also have some other requirements:

- Non-functional requirement: The new database tables must not use STI.
- Functional requirement: We need to be able to paginate.

Notification settings

Currently notification settings allow users to define highly customizable rules for when to receive email notifications. To create parity between the current to-dos and email system, we should add the ability for users to establish if they want to receive email only, email and web-based notification, or just an web-based notification.

NOTE: Changes to the notification settings system apart from adding notification/email differentiation are out of scope for this project.

New columns in the notificationsettingstable:

```
mermaid
erDiagram
    NOTIFICATIONSETTINGS {
        smallint channel
    }
```

This will allow to not change anything for current records (we will add the channel column with email value) and for web-based notifications - we will be adding separate row, allowing maximal flexibility.

Events

The event system is the backbone of this proposal. GitLab event store implementation is described here. Every notification record and every notification email should be handled in the subscriber to the particular event.

```
mermaid
flowchart TD
    A[System Actions] --> B[Event Store]
    B --> C[Event Processors]
    C --> D[Notification Service]
```

D --> E[Notification Database]
E --> F[API Layer]
F --> G[Delivery Channels]
G --> H[Email]
G --> K[Future Channels]

F --> L[User Interface]

N[User Preferences] --> D

The Event Store will serve as the single source of truth for all notification-triggering events. This approach provides several key advantages:

- Decoupling: System actions generate events without knowledge of how those events will be processed or which notifications they'll trigger.
- Consistency: Using a single event source ensures all notification types derive from the same consistent data.

Event Processing Pipeline

- Event Emission: When a relevant action occurs in the system (assignment, comment, due date change, etc.), an event is published to the Event Store.
- Event Processing: Event processors subscribe to specific event types and transform raw events into notification candidates.
- Notification Filtering: The notification service applies user preferences to determine if a notification should be created.
- Notification Storage: Valid notifications are stored in the unified notification database with appropriate metadata.
- Delivery Determination: Based on user preferences and notification type, the system determines which delivery channels to use (emails for now).

REST API and GraphQL endpoints

Important part of the new implementation is providing REST API and GraphQL endpoints that would allow to interact with notification records, in individual and batch manner. Current implementation of REST API and GraphQL endpoints for to-dos can serve as example of what we are looking for.

Alternative Solutions

1. Enhance current systems independently

Pros:

- Less initial development effort
- Lower migration risk
- Can be implemented incrementally

Cons:

- Maintains fragmented user experience
- Doesn't solve code maintenance issues
- Limited future extensibility
- Duplicated effort for common features

1. Introduce new notifications system with new settings, without use of the events

Pros:

- More flexibility on the settings side

Cons:

- Doesn't solve code maintenance issues
- Limited future extensibility

1. Introducing Read Model to Notifications table to not use STI pattern

Pros:

- Focus on retrieval patterns, optimizing for read speed

Cons:

- Requires extensive event system development to make sure every change is reflected in the notifications table

Architectural Decision Record

Database proposal: [link](#)

SECTION: engineering/architecture/design-documents/notifications/adr/001_database_schema.md

Proposal of database structure

- Technical Proposal: Notification System Design
- Goal

Design a flexible and normalized database structure to manage user notifications about different types of resources (issues, notes, merge requests, epics), while avoiding:

- Single Table Inheritance (STI)
- Polymorphic Associations

·· Database Design Overview

1. notifications table (centralized)

Stores notifications per user.

```
sql
CREATE TABLE notifications (
  id SERIAL,
  userid BIGINT NOT NULL REFERENCES users(id),
  read BOOLEAN DEFAULT FALSE,
  createdat TIMESTAMP DEFAULT NOW(),
  updatedat TIMESTAMP DEFAULT NOW(),
  namespaceid BIGINT NOT NULL REFERENCES namespaces(id),
  snoozeduntil TIMESTAMP,
  saved BOOLEAN DEFAULT FALSE,
  resolvedbyaction SMALLINT,
  authorid BIGINT REFERENCES users(id),
  action SMALLINT,
  resourcetype SMALLINT,
  PRIMARY KEY (id, userid)
) PARTITION BY HASH (userid);
```

This table should be partitioned using hash-based strategy. We should use 32 partitions (which would give us enough headway to accommodate future growth of this table). Userid column should be used as partition key, since lookup by userid is the most used usecase we should optimize for.

2. Resource Link Tables (one per resource)

Each notification links to exactly one resource via a dedicated table.

```
sql
CREATE TABLE issuenotifications (
  notificationid BIGINT NOT NULL,
  resourceid BIGINT NOT NULL REFERENCES issues(id) ON DELETE CASCADE,
  namespaceid BIGINT NOT NULL REFERENCES namespaces(id) ON DELETE CASCADE,
  userid BIGINT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  FOREIGN KEY (notificationid, userid) REFERENCES notifications(id, userid) ON DELETE CASCADE
);
```

```
CREATE TABLE notenotifications (
  notificationid BIGINT NOT NULL,
  resourceid BIGINT NOT NULL REFERENCES notes(id) ON DELETE CASCADE,
  namespaceid BIGINT NOT NULL REFERENCES namespaces(id) ON DELETE CASCADE,
  userid BIGINT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  FOREIGN KEY (notificationid, userid) REFERENCES notifications(id, userid) ON DELETE CASCADE
);
```

```
CREATE TABLE mergerequestnotifications (
  notificationid BIGINT NOT NULL,
  resourceid BIGINT NOT NULL REFERENCES mergerequests(id) ON DELETE CASCADE,
  namespaceid BIGINT NOT NULL REFERENCES namespaces(id) ON DELETE CASCADE,
```

```

    userid BIGINT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    FOREIGN KEY (notificationid, userid) REFERENCES notifications(id, userid) ON DELETE CASCADE
);

```

```

CREATE TABLE sshkeysnotificationlinks (
  notificationid BIGINT NOT NULL,
  resourceid BIGINT NOT NULL REFERENCES keys(id) ON DELETE CASCADE,
  namespaceid BIGINT NOT NULL REFERENCES namespaces(id) ON DELETE CASCADE,
  userid BIGINT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  FOREIGN KEY (notificationid, userid) REFERENCES notifications(id, userid) ON DELETE CASCADE
)

```

```

CREATE TABLE commitnotifications (
  notificationid BIGINT NOT NULL,
  resourceid BIGINT NOT NULL,
  namespaceid BIGINT NOT NULL REFERENCES namespaces(id) ON DELETE CASCADE,
  userid BIGINT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  FOREIGN KEY (notificationid, userid) REFERENCES notifications(id, userid) ON DELETE CASCADE
)

```

Namespaceid column should be that same as in the referenced notifications table, to be used as a sharding key.

Issuenotifications table will serve all workitem types, including epics and OKRs. We will use workitemtype field in issues table to differentiate between types.

Example queries with plans are listed in this snippet.

- Entity Relationship Diagram

```

mermaid
erDiagram
    users ||--o{ notifications : has
    notifications ||--|| issuenotifications : links
    notifications ||--|| notenotifications : links
    notifications ||--|| mergerequestnotifications : links

```

- Validation Strategy

- Application-Level Validation (Rails)

```

ruby
class Notification < ApplicationRecord
  belongs_to :user

  has_one :issuenotification
  has_one :notenotification
  has_one :mergerequestnotification
  <etc>

```

```

hasone :issue, through: :issuenotification
hasone :note, through: :notenotification
hasone :mergerequest, through: :mergerequestnotification
hasone :epic, through: :issuenotification
<etc>

```

```

validate :onlyonereresourcelinked

```

```

def onlyonereresourcelinked
  links = [
    issuenotification,
    notenotification,
    mergerequestnotification
  ].compact

```

```

    errors.add(:base, "Only one resource can be linked to a notification") if links.size > 1
  end
end

```

Notification Creation Service

Encapsulates logic for resource-safe creation:

```

ruby
class NotificationCreator
  def self.createfor(resource:, user:)
    Notification.transaction do
      notification = Notification.create!(user: user)

      case resource
      when Issue
        IssueNotification.create!(notification: notification, resource: resource)
      when Note
        NoteNotification.create!(notification: notification, resource: resource)
      when MergeRequest
        MergeRequestNotification.create!(notification: notification, resource: resource)
      when Epic
        IssueNotification.create!(notification: notification, resource: resource) it's because
we would use work-item approach, where every epic has a row in issues table
      else
        raise ArgumentError, "Unsupported resource type"
      end

      notification
    end
  end
end

```

- Rails Model Summary

Each link table has a corresponding model, e.g.:

```
ruby
class IssueNotification < ApplicationRecord
  belongsto :notification
  belongsto :resource
end
```

Repeat similarly for NoteNotification, MergeRequestNotification, and NoteNotification etc.

- Query Examples

Get all user notifications with resource type

```
sql
SELECT n.id, n.resourcetype, i.title, n.read, n.createdat
FROM notifications n
JOIN issuenotifications I ON I.notificationid = n.id
JOIN issues i ON i.id = I.resourceid
WHERE n.userid = :userid
```

UNION ALL

```
SELECT n.id, n.resourcetype, no.content, n.read, n.createdat
FROM notifications n
JOIN notenotifications I ON I.notificationid = n.id
JOIN notes no ON no.id = I.resourceid
WHERE n.userid = :userid
```

UNION ALL

```
SELECT n.id, n.resourcetype, mr.title, n.read, n.createdat
FROM notifications n
JOIN mergerequestnotifications I ON I.notificationid = n.id
JOIN mergerequests mr ON mr.id = I.resourceid
WHERE n.userid = :userid
```

UNION ALL

```
SELECT n.id, n.resourcetype, e.title, n.read, n.createdat
FROM notifications n
JOIN issuenotifications I ON I.notificationid = n.id
JOIN epics e ON e.issueid = I.resourceid
WHERE n.userid = :userid
```

ORDER BY createdat DESC;

Benefits of this design

- No STI or polymorphic associations
- Full referential integrity via FK constraints
- Clear separation of responsibilities
- Rails-friendly with explicit models
- Easier indexing and performance optimization
- Build-in partitioning, allowing us to store more rows than we store now

Challenges of this design

- Joining multiple tables at once
- Need for the careful queries structure to avoid inefficient queries

Exploring this design

Whilst not connected to this domain (notifications), this architectural design proposition was explored by the Workflow Catalog group in merge request 194032.

Rather than the notification creation service proposed above, we explored using a concern to include:

- `accepts_nested_attributes_for` - enables nested attribute handling
- `after_initialize` callback - creates the generic model automatically
- `delegate` and `assign_attributes` - handles reading and writing generic attributes

We would encourage evaluating this approach when implementing the new notifications architecture.

SECTION: engineering/architecture/design-documents/object_pools/_index.md

{{< engineering/design-document-header >}}

Summary

Forking repositories is at the heart of many modern workflows for projects hosted in GitLab. As most of the objects between a fork and its upstream project will typically be the same, this opens up potential for optimizations:

- Creating forks can theoretically be lightning fast if we reuse much of the parts of the upstream repository.
- We can save on storage space by deduplicating objects which are shared.

This architecture is currently implemented with object pools which hold objects of the primary repository. But the design of object pools has organically grown and is nowadays showing its limits.

This blueprint explores how we can iterate on the design of object pools to fix long standing issues with it. Furthermore, the intent is to arrive at a design that lets us iterate more readily on the exact implementation details of object pools.

Motivation

The current design of object pools is showing problems with scalability in various different ways. For a large part the problems come from the fact that object pools have organically grown and that we learned as we went by.

It is proving hard to fix the overall design of object pools because there is no clear ownership. While Gitaly provides the low-level building blocks to make them work, it does not have enough control over them to be able to iterate on their implementation details.

There are thus two major goals: taking ownership of object pools so that it becomes easier to iterate on the design, and fixing scalability issues once we can iterate.

Lifecycle ownership

While Gitaly provides the interfaces to manage object pools, the actual life cycle of them is controlled by the client. A typical lifecycle of an object pool looks as following:

1. An object pool is created via `CreateObjectPool()`. The caller provides the path where the object pool shall be created as well as the origin repository from which the repository shall be created.
1. The origin repository needs to be linked to the object pool explicitly by calling `LinkRepositoryToObjectPool()`.
1. The object pool needs to be regularly updated via `FetchIntoObjectPool()` that fetches all changes from the primary pool member into the object pool.
1. To create forks, the client needs to call `CreateFork()` followed by `LinkRepositoryToObjectPool()`.
1. Repositories of forks are unlinked by calling `DisconnectGitAlternates()`. This will reduplicate objects.
1. The object pool is deleted via `DeleteObjectPool()`.

This lifecycle is complex and leaks a lot of implementation details to the caller. This was originally done in part to give the Rails side control and

management over Git object visibility. GitLab project visibility rules are complex and not a Gitaly concern. By exposing these details Rails can control when pool membership links are created and broken. It is not clear at the current point in time how the complete system works and its limits are not explicitly documented.

In addition to the complexity of the lifecycle we also have multiple sources of truth for pool membership. Gitaly never tracks the set of members of a pool repository but can only tell for a specific repository that it is part of said pool. Consequently, Rails is forced to maintain this information in a database, but it is hard to maintain that information without becoming stale.

Repository maintenance

Related to the lifecycle ownership issues is the issue of repository maintenance. As mentioned, keeping an object pool up to date requires regular calls to `FetchIntoObjectPool()`. This is leaking implementation details to the client, but was done to give the client control over syncing the primary repository with its object pool. With this control, private repositories can be prevented from syncing and consequently leaking objects to other repositories in the fork network.

We have had good success with moving repository maintenance into Gitaly so that clients do not need to know about on-disk details. Ideally, we would do the same for repositories that are the primary member of an object pool: if we optimize its on-disk state, we will also automatically update the object pool.

There are two issues that keep us from doing so:

- Gitaly does not know about the relationship between an object pool and its members.
- Updating object pools is expensive.

By making Gitaly the single source of truth for object pool memberships we would be in a position to fix both issues.

Fast forking

In the current implementation, Rails first invokes `CreateFork()` which results in a complete git-clone(1) being performed to generate the fork repository. This is followed by `LinkRepositoryToObjectPool()` to link the fork with the object pool. It is not until housekeeping is performed on the fork repository that objects are deduplicated. This is not only leaking implementation details to clients, but it also keeps us from reaping the full potential benefit of object pools.

In particular, creating forks is a lot slower than it could be since a clone is always performed before linking. If the steps of creating the fork and linking the fork to the pool repository were unified, the initial clone could be

avoided.

Clustered object pools

Gitaly Cluster and object pools development overlapped. Consequently they are known to not work well together. Praefect does neither ensure that repositories with object pools have their object pools present on all nodes, nor does it ensure that object pools are in a known state. If at all, object pools only work by chance.

The current state has led to cases where object pools were missing or had different contents per node. This can result in inconsistently observed state in object pool members and writes that depend on the object pool's contents to fail.

One way object pools might be handled for clustered Gitaly could be to have the pool repositories duplicated on nodes that contain repositories dependent on them. This would allow members of a fork network to exist of different nodes. To make this work, repository replication would have to be aware of object pools and know when it needs to duplicate them onto a particular node.

Requirements

There are a set of requirements and invariants that must be given for any particular solution.

Private upstream repositories should not leak objects to forks

When a project has a visibility setting that is not public, the objects in the repository should not be fetched into an object pool. An object pool should only ever contain objects from the upstream repository that were at one point public. This prevents private upstream repositories from having objects leaked to forks through a shared object pool.

Forks cannot sneak objects into upstream projects

It should not be possible to make objects uploaded in a fork repository accessible in the upstream repository via a shared object pool. Otherwise potentially unauthorized users would be able to "sneak in" objects into repositories by simply forking them.

Despite leading to confusion, this could also serve as a mechanism to corrupt upstream repositories by introducing objects that are known to be broken.

Object pool lifetime exceeds upstream repository lifetime

If the upstream repository gets deleted, its object pool should remain in place to provide continued deduplication of shared objects between the other repositories in the fork network. Thus it can be said that the lifetime of the object pool is longer than the lifetime of the upstream repository. An object

pool should only be deleted if there are no longer any repositories referencing it.

Object lifetime

By deduplicating objects in a fork network, repositories become dependent on the object pool. Missing objects in the pooled repository cou